Software Documentation

PARMCO Project — Checkpoint 3
AI-Generated Documentation
Subsystem: RP4 control software (C and ARM assembly)

Table of Contents

1. Overview 1
2. Software Architecture 2
3. Library Attribution 2
4. How Each Library Works 3
4.1 Class library E4235 (instructor-provided) 3
4.2 Linux I2C interface and the AS5600 encoder 3
4.3 BlueZ and RFCOMM 4
4.4 POSIX threads 4
4.5 Signals, errno, time.h, math.h and file I/O 4
5. Code Walkthrough 4
5.1 Commands from the app 4
5.2 Telemetry to the app 5
5.3 Functions 6
5.4 Motor control loop 6
5.5 Encoder angle and RPM 7
5.6 IR RPM 7
5.7 CSV log 7
6. Build and Run 7
7. Code Flags (Human-Edit Markers) 8

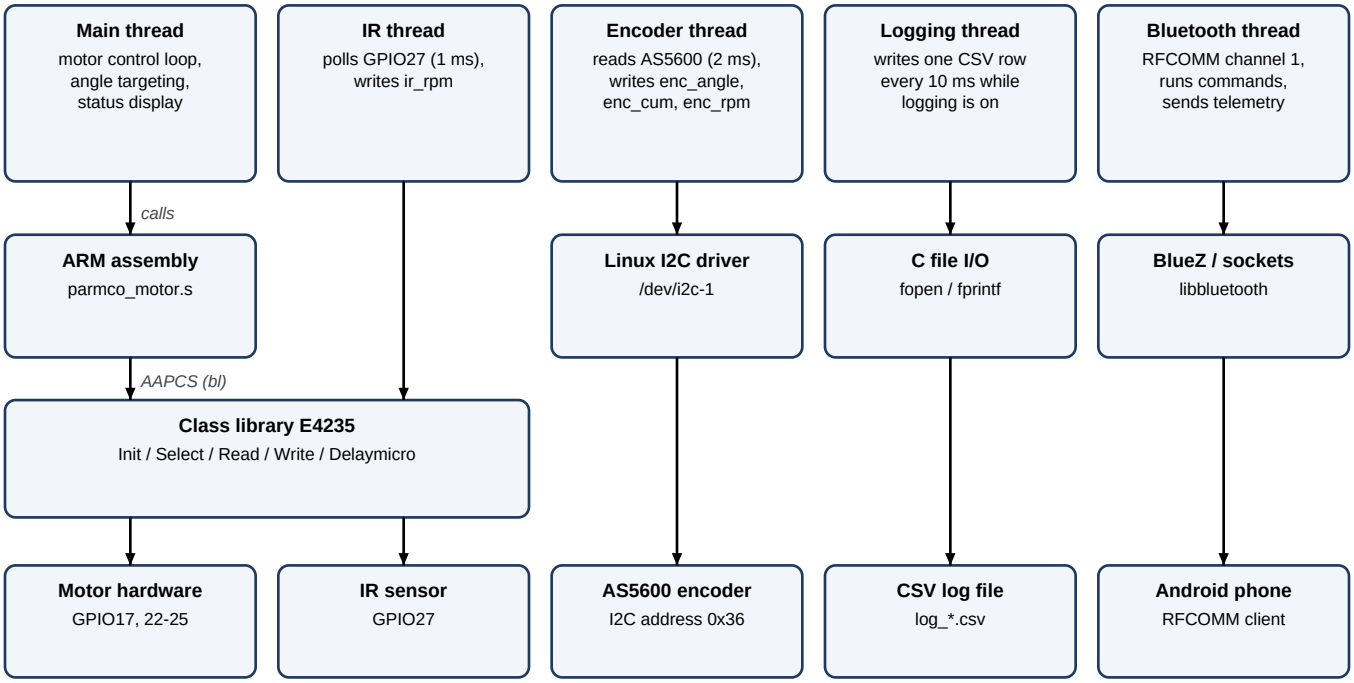
1. Overview

This document describes the Checkpoint 3 RP4 software: `main.c` (C: Bluetooth command server, motor control loop, AS5600 encoder reading, IR RPM measurement, CSV logging) and `parmco_motor.s` (ARMv7 AArch32 assembly: H-bridge enable routine, step routine, step-timing wrapper). Both were generated by an AI assistant; the class RP4 library (`E4235`) is provided by the instructor and is not AI-written. At Checkpoint 3 the phone app is the only control input: the RP4 receives text commands over Bluetooth, drives the motor, and sends telemetry back to the app every 0.15 s. Logging is started and stopped from the app. The program starts with the motor disabled and the default settings (Manual mode, clockwise, 20 RPM, motor stopped, not calibrated), prints a status line once per second for bench monitoring, and stops cleanly, with the motor disabled, when it receives Ctrl+C or a termination signal.

Hardware connections used by this code (BCM GPIO numbers): GPIO17 motor enable (via the 4N25), GPIO22, 23, 24, 25 coil inputs 1A, 2A, 3A, 4A of the SN754410NE, GPIO27 IR sensor input, and the RP4's I2C bus 1 (SDA on GPIO2, header pin 3; SCL on GPIO3, header pin 5) to the AS5600 magnetic encoder at I2C address 0x36. See the hardware documentation for the circuit.

2. Software Architecture

The program runs five threads: the main thread and four worker threads. The main thread owns the motor: it steps the motor, counts steps, performs angle targeting and prints the status line. The IR thread measures propeller RPM. The encoder thread reads the encoder angle and derives encoder RPM. The logging thread writes the CSV log. The Bluetooth thread receives the app's commands, updates the shared state, and sends telemetry. The threads communicate through shared global variables. After initialization, the motor's enable and step signals reach the hardware only through the assembly routines, and all GPIO access goes through the class library.



3. Library Attribution

Every library the code uses is listed here with its source, what it provides, and why the project needs it.

Library / header	Provider (attribution)	Used for in this code	Why it is needed
E4235 class library (`E4235.h`)	ECSE 4235 instructor. Not written by AI; the AI uses it.	E4235_Init, E4235_Select, E4235_Read, E4235_Write, E4235_Delaymicro	Project rule: all GPIO access, from C or assembly, goes through the class library. Assembly never touches GPIO registers directly.
Linux I2C device interface (`linux/i2c-dev.h`, `sys/ioctl.h`)	Linux kernel (kernel.org). Open source.	open("/dev/i2c-1"), ioctl(I2C_SLAVE), read, write	Reads the angle of the AS5600 magnetic encoder over I2C.
AS5600 magnetic encoder (device, not a library)	ams OSRAM AS5600 12-bit magnetic rotary position sensor; register information from its datasheet.	Angle register 0x0E-0x0F at I2C address 0x36	Gives the shaft angle used for set to zero, angle targets and encoder RPM.
BlueZ (`bluetooth/bluetooth.h`, `bluetooth/rfcomm.h`, libbluetooth)	BlueZ project, the official Linux Bluetooth stack (bluez.org). Open source.	sockaddr_rc, AF_BLUETOOTH, BTPROTO_RFCOMM, BDADDR_ANY	Gives C a serial-style Bluetooth link to the Android app.

Library / header	Provider (attribution)	Used for in this code	Why it is needed
POSIX threads (`pthread.h`)	POSIX threads in the GNU C Library (glibc) on Raspberry Pi OS.	pthread_create, pthread_join	Runs IR sensing, encoder reading, logging and the Bluetooth server beside the motor loop so none of them blocks the others.
Signals and error codes (`signal.h`, `errno.h`)	glibc / POSIX.	signal, SIGINT, SIGTERM, SIGPIPE; errno, EAGAIN, EWOULDBLOCK	Ctrl+C or a termination request ends the program cleanly with the motor disabled; a closed Bluetooth link cannot kill the program; "no data yet" is told apart from a real socket error.
POSIX/Linux system calls (`unistd.h`, `fcntl.h`, `sys/socket.h`)	glibc / Linux kernel.	read, write, close, usleep, fcntl (O_NONBLOCK), socket, setsockopt, bind, listen, accept	Device and socket I/O, non-blocking mode, and sleeping between polls.
C math (`math.h`, libm)	glibc.	fabs, fabsf, fmodf	Absolute values of angle changes and errors, and wrapping an angle target into 0 to 360 degrees.
C time (`time.h`)	glibc.	clock_gettime(CLOCK_MONOTONIC), time, localtime, strftime	Monotonic timing for IR pulses, encoder RPM, telemetry and log timestamps; the date and time in log file names.
C standard library (`stdio.h`, `stdlib.h`, `string.h`)	glibc.	printf, perror, fopen, fprintf, fflush, snprintf, atof, memset, strncmp, strcpy, strlen, strtok_r	Log file output, command parsing, number conversion and telemetry formatting.

The header `termios.h` is included in `main.c`, but none of its functions are called, because the program has no keyboard input.

4. How Each Library Works

4.1 Class library E4235 (instructor-provided)

This section describes the library interface as the code uses it. The call pattern is: `E4235_Init()` once at start-up; `E4235_Select(pin, mode)` sets a pin as an output (mode 1) or input (mode 0); `E4235_Write(pin, value)` drives an output to 0 or 1; `E4235_Read(pin)` returns the level of an input; `E4235_Delaymicro(us)` waits for the given number of microseconds. Pin numbers are BCM GPIO numbers. The C code calls Init and Select, and the IR thread calls Read; the assembly calls `E4235_Write` and `E4235_Delaymicro` with `bl` using the ARM calling convention (AAPCS): first argument in r0, second in r1.

4.2 Linux I2C interface and the AS5600 encoder

I2C is a two-wire bus (SDA and SCL) on which the RP4 is the master and each device has a 7-bit address. Linux exposes the RP4's bus 1 as the file `/dev/i2c-1`, which exists once I2C is enabled in the operating system's configuration. The program opens the file, then `ioctl(fd, I2C_SLAVE, 0x36)` selects the AS5600 (default address 0x36) as the target of later reads and writes. If either step fails, the thread prints the error and ends. To read the angle the thread writes one byte, the register number 0x0E, then reads two bytes: the high and low bytes of the 12-bit angle (0 to 4095 counts for one revolution). The thread combines them as `((high & 0x0F) << 8) | low`, keeping only the 12 valid bits, and multiplies by 360/4096 = 0.087890625 to get degrees. The encoder reports its angle relative to how its magnet sits on the shaft, so the program subtracts a stored zero offset (set by calibrate zero) and wraps the result into 0 to 360 degrees. The constant `ENCODER_INVERT_DIR` (0 in this version) is set to 1 at compile time if the encoder angle decreases when the motor turns clockwise; the angle is then reversed as 360 minus the reading.

4.3 BlueZ and RFCOMM

RFCOMM emulates a serial port over a Bluetooth link. BlueZ exposes it to C as an ordinary stream socket, so the server uses the same call order as a TCP server: `socket(AF_BLUETOOTH, SOCK_STREAM, BTPROTO_RFCOMM)`, `setsockopt(SO_REUSEADDR)` so the program can be restarted immediately, `bind` to a `sockaddr_rc` (any local adapter, channel 1), `listen`, then `accept`. If the socket cannot be created or bound, the thread prints the error and ends. After the phone connects, `read` returns the command bytes it sent and `write` sends telemetry back. Both the listening socket and the client socket are set non-blocking with `fcntl(..., O_NONBLOCK)`, so `accept` and `read` return immediately when nothing is waiting; the thread polls the client every 10 ms and polls for a new client every 100 ms. When `read` returns 0 (the phone closed the link), returns an error other than "no data yet", or a telemetry `write` fails, the thread closes the client socket and goes back to waiting for a new connection, so the phone can reconnect without restarting the program. Pairing and remembering the connection are done by the BlueZ background service and the phone, not by this program; BlueZ stores the pairing keys on the RP4. The code does not register a service record (SDP), so the phone app connects to RFCOMM channel 1 directly, or a Serial Port service record is registered separately.

4.4 POSIX threads

`pthread_create` starts a function in a new thread that shares the program's memory; `pthread_join` waits for a thread to finish. The threads share global variables, each written mostly by one thread: the encoder thread writes `raw_enc_deg`, `enc_angle`, `enc_cum_angle` and `enc_rpm`; the IR thread writes `ir_rpm`; the main thread writes `step_count` and `step_angle`; the Bluetooth thread writes the command state (`is_running`, `direction`, `cmd_rpm`, `current_mode`, `overrange_unlock`, the target and calibration variables and `logging_enabled`). There is no mutex: each variable holds one 32-bit or 64-bit number. The `volatile` keyword makes each thread re-read a variable instead of caching it. At shutdown the main thread joins all four worker threads.

4.5 Signals, errno, time.h, math.h and file I/O

`signal(SIGINT, handle_sigint)` and `signal(SIGTERM, handle_sigint)` make Ctrl+C and a termination request call a small handler that sets `app_running` to 0; every thread loop ends, and the main thread disables the motor and joins the threads. `signal(SIGPIPE, SIG_IGN)` stops a write to a closed Bluetooth socket from killing the program; the write returns an error instead. `errno` holds the reason for the last failed call, and the code compares it with `EAGAIN` and `EWOULDBLOCK`, which mean only that a non-blocking read or write had nothing to do. `clock_gettime(CLOCK_MONOTONIC, ...)` returns seconds and nanoseconds since an arbitrary start that never goes backwards; the code subtracts two readings to get elapsed seconds. `time`, `localtime` and `strftime` turn the wall-clock time into the log file name. `fopen`, `fprintf` and `fflush` write the CSV file; the logging thread flushes after every row so the file is complete even if the program stops. `fabs` and `fabsf` take absolute values, and `fmodf` gives the remainder of a division, used to wrap an angle target.

5. Code Walkthrough

5.1 Commands from the app

The Bluetooth thread splits what it reads at line endings, so several commands in one packet are each run in order. Every command is printed to the terminal as it is run, and is matched by its leading text.

Message	Effect
`CMD:START`	target_active = 0, is_running = 1, motor enabled (GPIO17 high through enable_motor(1)).
`CMD:STOP`	is_running = 0, target_active = 0, motor disabled (enable_motor(0)) so the coils carry no current, and the over-range unlock is switched off.
`DIR:CW` / `DIR:CCW`	direction = +1 (clockwise) / -1 (counterclockwise).

Message	Effect
`CMD:FASTER`	cmd_rpm x 1.05. Limited to 100 RPM unless the over-range unlock is on.
`CMD:SLOWER`	cmd_rpm x 0.95. Limited to a minimum of 10 RPM.
`UNLOCK:ON` / `UNLOCK:OFF` (or `OVERRANGE:ON` / `OVERRANGE:OFF`)	Switches the over-range unlock on / off.
`RPM:<value>`	cmd_rpm = the number after the colon if it is 10 or more; a value below 10 is ignored. The RP4 does not check the 100 RPM upper limit for this command.
`MODE:MANUAL` / `MODE:MAINTAIN`	current_mode = 0 / 1. The mode is recorded in the log; the motor control in this version does not use it.
`CALIBRATE:ZERO`	Stores the latest raw encoder angle (from the encoder thread) as zero_offset_deg; sets enc_angle, enc_cum_angle, step_count and step_angle to 0; sets is_calibrated = 1.
`SET:ZERO`	If calibrated: target angle 0, is_running = 0, target_active = 1, motor enabled. If not calibrated: ignored, with a warning on the terminal.
`ANGLE:<degrees>`	If calibrated: the target is the number after the colon wrapped into 0 to 360 degrees, is_running = 0, target_active = 1, motor enabled. If not calibrated: ignored, with a warning on the terminal.
`LOG:START` / `LOG:STOP`	logging_enabled = 1 / 0.

5.2 Telemetry to the app

Every 0.15 s while a phone is connected the Bluetooth thread sends one line: `TEL:ir_rpm,enc_rpm,enc_angle,calibration,log_file,elapsed`, for example `TEL:20.1,19.8,123.4,Calibrated,log_2026-11-12_10-15-00.csv,12s`. The three numbers have one decimal place; the calibration field is "Calibrated" or "Not Calibrated"; the log file field is the current log file name, or "none" when logging is off; the last field is the log's elapsed time in whole seconds.

5.3 Functions

Function	File	What it does
main	main.c	Installs the signal handlers, initializes the library, sets GPIO17, 22-25 as outputs, calls enable_motor(0) so the motor starts disabled, prints the step mode (half step, 400 steps per revolution) and the start-up state, starts the IR, encoder, logging and Bluetooth threads, then runs the motor control loop. On exit it disables the motor and joins the threads.
handle_sigint	main.c	Sets app_running to 0 when Ctrl+C or a termination signal arrives.
get_shortest_path	main.c	Returns target minus current angle, wrapped into -180 to +180 degrees, so a move always takes the shorter way around.
ir_sensor_thread	main.c	Sets GPIO27 as input and polls it every 1 ms. On a low-to-high change it measures the time since the previous pulse and updates ir_rpm (section 5.6). If no pulse arrives for 1.8 s it sets ir_rpm to 0.
encoder_thread	main.c	Every 2 ms reads the AS5600 angle, applies the zero offset, updates enc_angle, updates enc_cum_angle, and updates enc_rpm every 0.1 s (section 5.5).
logging_thread	main.c	Opens a new CSV file when logging is switched on, writes a row every 10 ms, and closes the file when logging is switched off. It prints a message to the terminal when logging starts and stops.
process_bt_command	main.c	Prints and runs one command string from the app (section 5.1).
bt_server_thread	main.c	RFCOMM server on channel 1. Accepts a phone, passes each received line to process_bt_command, sends the telemetry line in section 5.2, and returns to waiting when the phone disconnects.
enable_motor(state)	parmco_motor.s	Moves state into r1, loads 17 into r0, calls E4235_Write(17, state).
step_motor(direction, mode)	parmco_motor.s	Adds direction to current_step_index, wraps it with AND 7, looks up the byte in half_step_table (0x08, 0x0C, 0x04, 0x06, 0x02, 0x03, 0x01, 0x09), and writes its four bits to GPIO22 (bit 3), GPIO23 (bit 2), GPIO24 (bit 1), GPIO25 (bit 0). The mode argument is accepted but ignored (half-step only).
wait_and_step(delay_us, direction)	parmco_motor.s	Calls E4235_Delaymicro(delay_us), then step_motor(direction, 0). The main loop calls it with direction +1 or -1.

5.4 Motor control loop

Once per second the main loop prints a status line (motor enable, commanded RPM, IR RPM, encoder RPM, encoder angle and calibration state) for bench monitoring; it is view-only. The loop sleeps for 10 ms per pass while the motor is not running. While the motor is enabled and either running or targeting an angle, each pass takes one step. If an angle target is active, it first computes the error with `get_shortest_path(enc_angle, target_angle_deg)`. When the error is within 0.9 degrees (one half step), the target is cleared, running stops, the motor is disabled so no coil current flows, and the terminal reports the position reached. Otherwise the step direction is toward the target (+1 clockwise, -1 counterclockwise), and within 5 degrees of the target the speed is limited to 12 RPM so the encoder updates between steps. The step rate is `rpm / 60 x 400` steps per second (half-step mode has 400 steps per revolution, so 20 RPM is 133.3 steps/s) and the delay is 1,000,000 / step rate microseconds; `wait_and_step` is then called with the step direction. The step count goes up or down by one and `step_angle` is step_count x 0.9 degrees. Speed in Manual mode is open loop: it comes from cmd_rpm only. Angle moves use the same cmd_rpm, so the desired RPM sets how fast the blade moves to its target.

5.5 Encoder angle and RPM

`enc_angle` is the offset-corrected angle in 0 to 360 degrees. `enc_cum_angle` is the unwrapped angle: each sample's change in angle is wrapped into -180 to +180 degrees and added, so it keeps counting past 360 and below 0. A change smaller than 0.15 degrees is ignored while the motor is stopped, which hides the encoder's one-count flicker at rest. Every 0.1 s the thread sets `enc_rpm` to the absolute value of the change in `enc_cum_angle` over that window, divided by 360, divided by the window time, times 60; when the motor is disabled and this value is below 1 RPM, `enc_rpm` is set to 0. Calibrate zero stores the raw encoder angle at that moment as the zero offset, so the blade must be aligned with 0 degrees on the protractor before it is pressed.

5.6 IR RPM

The propeller has two blades, so consecutive IR pulses are half a revolution apart and an instantaneous RPM = 60 / (2 x seconds between pulses) = 30 / seconds. Pulses closer together than 0.03 s are ignored as noise. The first reading after the speed was 0 is used as is; later readings are smoothed as new reading x 0.7 + previous value x 0.3, so the display settles quickly without jitter. The value returns to 0 when no pulse has arrived for 1.8 s.

5.7 CSV log

Each log is a file named `log_YYYY-MM-DD_HH-MM-SS.csv` created in the program's working directory. Rows are written about every 10 ms (100 Hz). The time column is seconds from the monotonic clock since logging started.

Column	Meaning
time_s	Seconds since logging started (monotonic clock)
mode	position while an angle target is active; otherwise Manual or Maintain
cmd_rpm	Commanded speed
cmd_angle_deg	Angle target (target_angle_deg)
step_count	Half steps commanded since the last calibrate zero (signed)
step_angle_deg	step_count x 0.9: where the RP4 thinks the shaft is
enc_angle_deg	Encoder angle after the zero offset, 0 to 360 degrees
enc_cum_deg	Encoder angle unwrapped (cumulative degrees)
ir_rpm	RPM from the IR sensor
enc_rpm	RPM derived from the encoder angle
app_rpm	The actual RPM sent to the app (the IR RPM)
enable	1 when the motor is enabled, 0 when disabled

6. Build and Run

The code needs the BlueZ development headers (on Raspberry Pi OS, the package `libbluetooth-dev`), the class library supplied by the instructor, and a 32-bit ARM toolchain (the assembly is ARMv7 AArch32). I2C must be enabled on the RP4 so that `/dev/i2c-1` exists. The two source files are compiled and linked together with the Bluetooth, thread and math libraries, and with the class library in the way the course instructions specify:

```
gcc -o parmco main.c parmco_motor.s -lbluetooth -lpthread -lm
```

For the phone link, the RP4's Bluetooth adapter must be powered, pairable and visible to the phone. The program has no keyboard input. At start-up it prints "Waiting for Android phone connection on RFCOMM Channel 1" once the Bluetooth server is listening, and it leaves the motor disabled. Press Ctrl+C to stop it; the motor is disabled before the program exits.

7. Code Flags (Human-Edit Markers)

The ground rules require every human change in code to be marked. Each change is opened with a `[HUMAN EDIT START]` comment that gives the reason, keeps the AI original as a comment beginning `AI original:`, and is closed with an `[END HUMAN EDIT]` comment. Both source files were scanned for these markers and none were found, so both files are the unmodified AI original.

File	Human edits
main.c and parmco_motor.s	None